TOPIC 5.1

Gradient Descent: Batch, Stochastic and Mini-batch

How machine-learning models find their best parameters, one small step at a time

WHAT YOU WILL LEARN

- Explain **why** models need an optimiser and what a **cost function** is
- Describe gradient descent as *walking downhill* and write the update rule **new = old − learning rate × slope**
- Work through a full example by hand: slope, step size, new value, and when to stop
- Choose a sensible **learning rate** and recognise too-small and too-large values
- Explain **batch**, **stochastic** and **mini-batch** gradient descent, and when each one is used
- Write gradient descent from scratch in a few lines of NumPy

Part A. The idea

1. Why do we need gradient descent?

Almost every model has **parameters**, numbers it must choose before it can predict. A straight line has two: the slope **m** and the intercept **b**. *Training* a model means finding parameter values that make its predictions as good as possible.

In the first class example the data follows the line **y = 2x + 3**. We pretend we do not know that and ask the computer to find it:

x	1	2	3	4	5
y	5	7	9	11	13

A person could guess a line, look at the gap to the data, and adjust. **Gradient descent is that same idea, made precise and automatic.** It is also not limited to lines: it trains logistic regression, neural networks and many other models, which is why it is one of the most important ideas in machine learning.

2. The cost function: how wrong is the line?

To improve a line we first need a number that says **how bad it is**. For each point, the **residual** is the observed value minus the predicted value. Squaring each residual makes all of them positive, and averaging gives the **mean squared error (MSE)**. In machine-learning language this number is the **cost function** (also called the loss function).

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - (m\,x_i + b))^2$$

DEFINITION **Cost function**

A single number that measures how far a model's predictions are from the real values. **Training = finding the parameters that make the cost as small as possible.**

The second class example uses house prices in Monroe Township, New Jersey:

Area (sq ft)	2600	3000	3200	3600	4000
Price ($)	550,000	565,000	610,000	680,000	725,000

WORKED EXAMPLE **Cost of a guess versus the best line**

- Guess: price = 100 × area + 250,000. Predictions: 510,000, 550,000, 570,000, 610,000, 650,000
- Residuals: 40,000, 15,000, 40,000, 70,000, 75,000. The MSE is the average of their squares: **2,790,000,000**
- The best straight line is price = 135.79 × area + 180,616. Its MSE is **186,815,068**, far smaller
- Gradient descent is the method that walks from the first line to the second without being told the answer

3. Walking downhill in the fog

Imagine standing on a hillside in thick fog, wanting to reach the valley floor. You cannot see the valley, but you can feel **which way the ground slopes under your feet**. So you take a step downhill, feel the slope again, and repeat. When the ground feels flat, you have arrived.

The hill is the cost function. Your position is the pair of parameter values (m, b). The slope under your feet is the **gradient**. That is the whole algorithm.

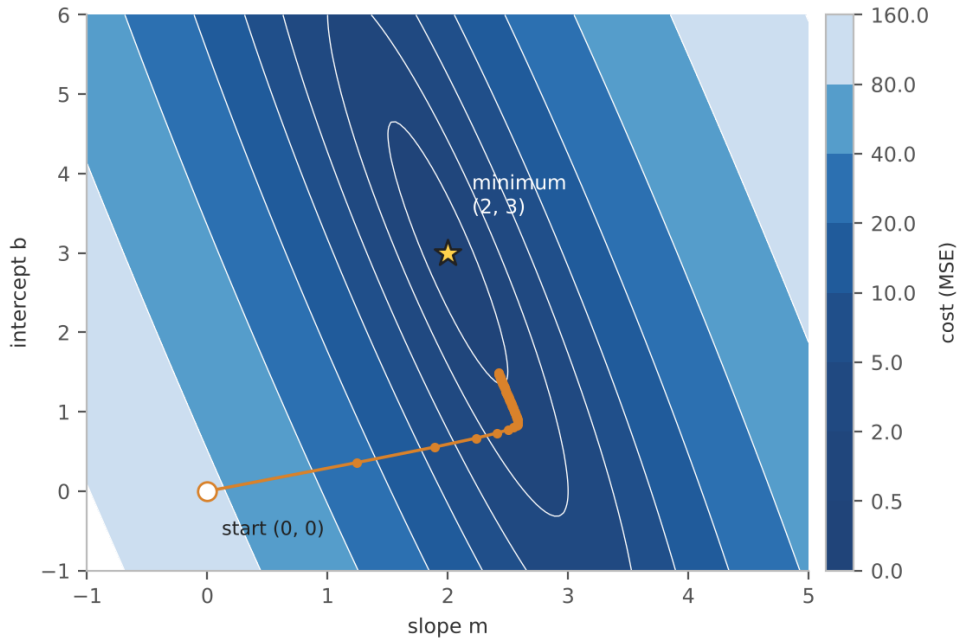


Figure 1. The cost of every possible line for the data y = 2x + 3. Darker means lower cost. Gradient descent starts at (0, 0) and walks towards the minimum at (2, 3): big steps first, then it crawls along the valley floor.

DEFINITION **Gradient descent**

An algorithm that finds the minimum of a cost function by repeating one move: measure the slope of the cost at the current parameter values, then move a small step in the **downhill** direction (opposite to the slope).

4. The update rule

Two ingredients decide each move:

- **The slope** (the derivative of the cost). If it is steep, we are far from the bottom and can take a big step. If it is nearly flat, we are close and should take a baby step.
- **The learning rate**, a small number you choose that scales every step.

$$\text{step size} = \text{slope} \times \text{learning rate} \qquad \text{new value} = \text{old value} - \text{step size}$$

The minus sign is what makes it *descent*. When the slope is negative (the cost falls as the parameter grows), subtracting a negative number **increases** the parameter, which is the right direction. When the slope is positive, the parameter decreases.

For the line y = m·x + b with the MSE cost, the slopes (partial derivatives) with respect to each parameter are:

$$\frac{\partial \text{MSE}}{\partial m} = -\frac{2}{n}\sum_i x_i(y_i - \hat{y}_i) \qquad \frac{\partial \text{MSE}}{\partial b} = -\frac{2}{n}\sum_i (y_i - \hat{y}_i)$$

Here ŷ is the line's prediction, m·x + b. Both parameters are updated **together** at every step.

Part B. A complete example by hand

To see every number, the class used the smallest possible problem: three people, with the slope fixed at 0.64 and **only the intercept** to be found.

Weight	0.5	2.3	2.9
Height	1.4	1.9	3.2

The line is *Predicted height = intercept + 0.64 × weight*, and the cost is the **sum of squared residuals** (SSR). We start from a random guess, **intercept = 0**, with learning rate **0.1**.

Step 1: the cost at the starting guess

With intercept 0 the predictions are 0.32, 1.47 and 1.86, so the residuals are 1.08, 0.43 and 1.34. The SSR is $1.08^2 + 0.43^2 + 1.34^2 =$ **3.16**. (The slides round the residuals to 1.1, 0.4 and 1.3 and quote about 3.1.)

Step 2: the slope of the cost at that point

Differentiating the SSR with respect to the intercept (using the chain rule on each squared term) gives **−2 × (sum of the residuals)**, one term for each of the three people:

$$\frac{d\,\text{SSR}}{d\,\text{intercept}} = -2\sum_{i=1}^{3}(\text{height}_i - (\text{intercept} + 0.64 \cdot \text{weight}_i))$$

At intercept 0 this equals **−5.7** (exactly −5.704). The negative slope says: raising the intercept lowers the cost.

Step 3: step size, then the new intercept

Step size = −5.7 × 0.1 = **−0.57**. New intercept = old intercept − step size = 0 − (−0.57) = **0.57**. In one big step the line moved much closer to the data. The slope is the largest here, so the step is the largest.

Step 4: repeat

Step	Intercept now	Slope of the cost	Step size (slope × 0.1)	New intercept
1	0.0000	−5.704	−0.5704	0.5704
2	0.5704	−2.282	−0.2282	0.7986
3	0.7986	−0.913	−0.0913	0.8898
4	0.8898	−0.365	−0.0365	0.9263
5	0.9263	−0.146	−0.0146	0.9409
6	0.9409	−0.058	−0.0058	0.9468
7	0.9468	−0.023	−0.0023	0.9491

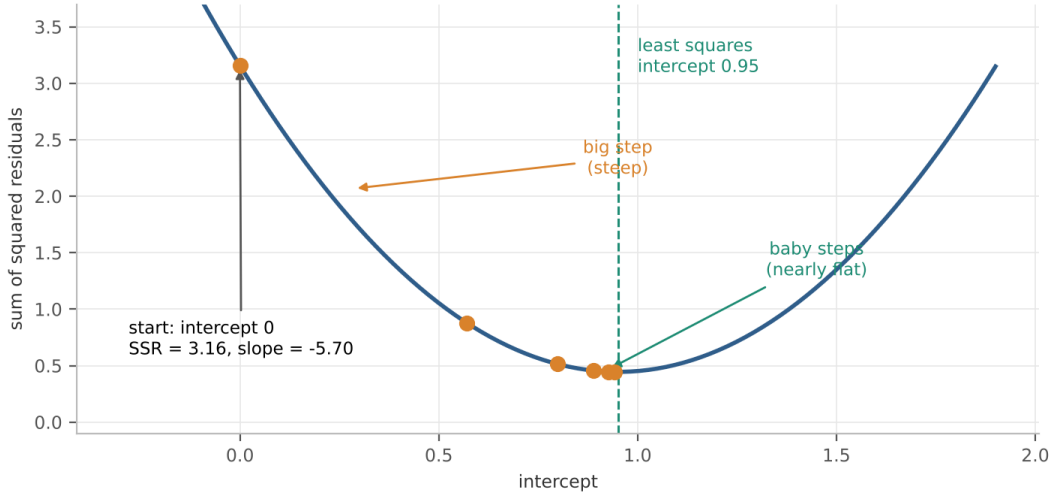


Figure 2. Each orange dot is one step. The first step is long because the slope is steep; the later steps shrink to almost nothing as the curve flattens near the bottom.

> **KEY IDEA** **Steps shrink on their own**
>
> Nobody tells gradient descent to slow down. Because **step size = slope × learning rate**, and the slope approaches 0 at the minimum, the steps get smaller automatically.

When does it stop?

Gradient descent never reaches the exact bottom, it creeps towards it. So we need stopping rules. Two are used together:

- **Minimum step size.** Stop when the step is smaller than a tiny number, in practice **0.001** or less. Here the 8th step would be −0.0009, which is below 0.001, so the algorithm stops after **7 steps** with intercept **0.9491**.
- **Maximum number of steps.** A safety limit, in practice about **1,000**, so a badly chosen learning rate cannot make it run forever.

For comparison, least squares (the exact formula) gives an intercept of 0.9507 with this slope, about **0.95**. Gradient descent found essentially the same answer by walking rather than by solving.

```
learning_rate = 0.1
min_step = 0.001                    # stop when the step is smaller than this
max_steps = 1000                    # ... or after this many steps

intercept = 0.0                     # initial guess
weight = [0.5, 2.3, 2.9]
height = [1.4, 1.9, 3.2]

for step in range(max_steps):
    slope = sum(-2 * (h - (intercept + 0.64 * w)) for w, h in zip(weight, height))
    step_size = slope * learning_rate
    if abs(step_size) < min_step:
        break
    intercept = intercept - step_size

print(step, round(intercept, 4))
```

Part C. Learning rate and other practical points

5. Both parameters at once: y = 2x + 3

Back to the first example. Now both m and b are unknown, we start at m = 0 and b = 0, and use learning rate 0.08. The two slope formulas from section 4 are applied together at every step.

Step	m	b	Cost (MSE)
0	0.00	0.00	89.00
1	4.96	1.44	71.11
2	0.50	0.27	56.83
3	4.45	1.43	45.44
4	0.89	0.50	36.35

The first step overshoots (m jumps to 4.96), the next steps correct it, and after about 400 steps the line is y = 2.00x + 3.00, exactly the line that generated the data.

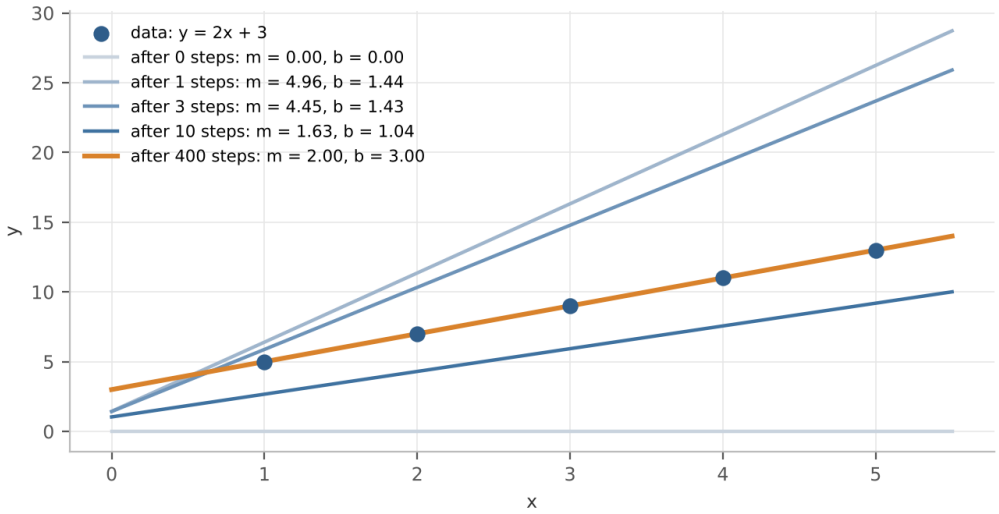


Figure 3. The line at different moments of training. The pale lines are the early, poor guesses; the orange line has settled on the data.

6. The learning rate

The learning rate is **not learned**; you choose it. It is a *hyperparameter*, and it matters a great deal:

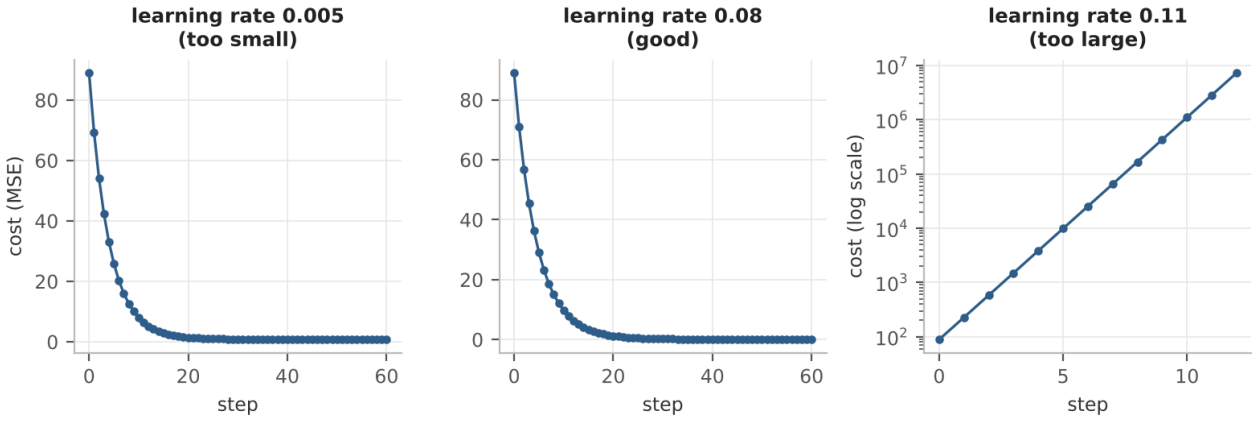


Figure 4. The same data and the same algorithm with three learning rates. Too small and it crawls; a good value converges quickly; too large and every step overshoots further, so the cost explodes (note the log scale in the right panel).

Learning rate	What happens	Looks like
Too small	Safe but very slow; may hit the step limit before arriving	Cost falls in tiny steps
Good	Converges in a reasonable number of steps	Cost falls fast, then levels off
Too large	Steps jump over the minimum, back and forth; the cost grows (diverges)	Cost goes **up**

KEY IDEA **A common strategy**

Start with a relatively **large** learning rate and make it **smaller** as training goes on. How the rate changes over time is called the **schedule**. Many libraries do this for you; in simple examples it is just fixed.

7. Local and global minima, and why scaling helps

- For **linear regression** the cost is a single smooth bowl (it has exactly one minimum), so gradient descent always finds the best line if the learning rate is reasonable. The class diagrams of a bumpy surface with a *global minimum* describe more complex models, such as neural networks, where the surface can have several dips (*local minima*) and the starting point matters.
- **Scale your features first.** If one feature is in the thousands (like house area) and another is between 0 and 1, the bowl becomes a long narrow valley, so one learning rate cannot suit both directions. On the house-price data, a learning rate of 0.01 that is fine for small numbers makes the cost **explode** with raw areas, while standardised areas converge calmly. The Scaling notes (5.6) explain how.

COMMON MISTAKE **Common mistakes**

- **Getting the sign wrong.** The update is *old* − *step*, not *old* + *step* (that would climb the hill)
- **Updating one parameter before computing the other's slope.** Compute both slopes first, then update both
- **Using the same learning rate for any data.** A rate that works on small numbers can blow up on large ones. Scale the features
- **Thinking a falling cost means success.** Check that it has *levelled off*, and that the result makes sense on the data

Part D. Batch, stochastic and mini-batch

8. The problem with using all the data at every step

In every example so far, each slope was computed from **all** the data points: three terms for the intercept, three for the slope. That is cheap. But real data sets have millions of rows, and real models have many parameters. The class example was a logistic regression with **23,000 genes** as inputs: every step needs a slope for each of the 23,000 parameters, and each slope adds up a term for every row. Millions of rows times 23,000 parameters, for *every single step*, takes a very long time.

9. Stochastic gradient descent (SGD)

DEFINITION **Stochastic gradient descent**

Gradient descent that uses **one randomly picked sample** per step, instead of the whole data set, to estimate the slope. (*Stochastic* means random.)

With one sample the sum disappears: the slope has a single term. With a million rows that is a million times less work per step. SGD works especially well when the data is **redundant**. The class picture had 12 points that form three tight clusters; any one point from a cluster tells you almost the same as the others.

WORKED EXAMPLE **One SGD step (class example)**

- Start: intercept = 0, slope = 1. A random sample is picked: weight 3, height 3.3
- Slope for the intercept: $-2 \times (3.3 - (0 + 1 \times 3)) = \mathbf{-0.6}$. Slope for the slope parameter: $-2 \times 3 \times (3.3 - (0 + 1 \times 3)) = \mathbf{-1.8}$
- Learning rate 0.01, so the step sizes are $-0.6 \times 0.01 = \mathbf{-0.006}$ and $-1.8 \times 0.01 = \mathbf{-0.018}$
- New intercept = $0 - (-0.006) = \mathbf{0.006}$. New slope = $1 - (-0.018) = \mathbf{1.018}$. Then pick another random point and repeat

Because each step looks at one point, the path is **noisy**: it zig-zags towards the minimum instead of sliding straight down, as Figure 5 shows.

10. Mini-batch gradient descent

The strict definition of SGD is one sample per step. In practice it is much more common to use a **small subset, called a mini-batch**, for each step, for example 3 samples (or 32 or 64 in real projects). A mini-batch takes the best of both worlds: **steadier** than one sample, and **much faster** than all the data.

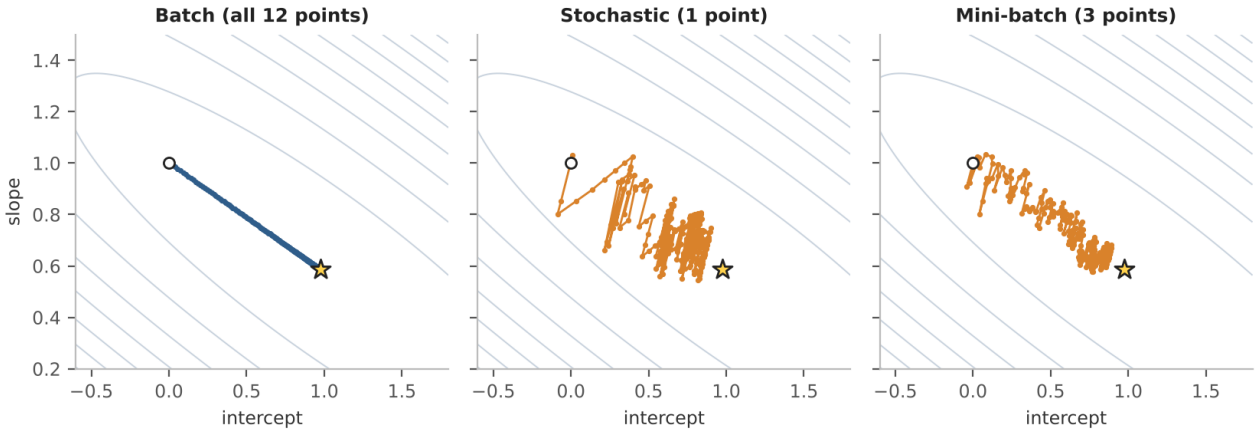


Figure 5. The same 12-point problem solved three ways, starting from the white dot. Batch walks straight to the star (the least-squares answer). Stochastic and mini-batch zig-zag around it; the mini-batch path is less jagged.

	Batch (all data)	Stochastic (1 sample)	Mini-batch (a few samples)
Data used per step	All rows	1 random row	A small random subset
Work per step	Largest	Smallest	Small
Path to the minimum	Smooth, direct	Very noisy	Slightly noisy
Best for	Small data sets	Huge, redundant data; streaming data	Most real projects (the usual default)

In this 12-point run (300 steps), least squares gives intercept 0.974 and slope 0.587. Batch gradient descent reached 0.971 and 0.588. Stochastic ended at 0.880 and 0.707, and mini-batch at 0.855 and 0.597. The noisy methods land *near* the answer, not exactly on it.

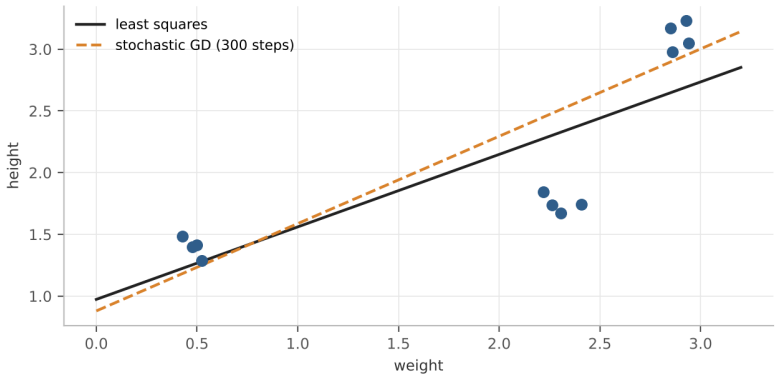


Figure 6. The 12 clustered points, the exact least-squares line, and the line that stochastic gradient descent ended with: close, but not identical.

11. A bonus: easy updates when new data arrives

With batch gradient descent, new data means starting again. With SGD you simply **pick up where you left off**: take one more step using the new sample, starting from the current slope and intercept (in the class example, from intercept 0.86 and slope 0.68), with no need to go back to the initial guess. This is why SGD suits data that keeps arriving.

Part E. In code

Here is gradient descent from scratch for y = 2x + 3 (batch version). The loop is exactly the update rule from section 4. Running it prints `2.0 3.0`.

```
import numpy as np

x = np.array([1, 2, 3, 4, 5], dtype=float)
y = 2 * x + 3                      # the true line: y = 2x + 3

m, b = 0.0, 0.0                    # start from a guess
learning_rate = 0.08

for step in range(1000):
    error = y - (m * x + b)                  # residuals
    grad_m = -2 * np.mean(x * error)         # slope of the cost along m
    grad_b = -2 * np.mean(error)             # slope of the cost along b
    m = m - learning_rate * grad_m           # new = old - learning rate * slope
    b = b - learning_rate * grad_b

print(round(m, 3), round(b, 3))
```

In scikit-learn, `SGDRegressor` and `SGDClassifier` implement stochastic gradient descent, and `partial_fit` updates a model with new batches of data. Linear models such as `LinearRegression` use a direct formula instead, which is why you rarely write gradient descent yourself for them, but deep-learning libraries use it for everything.

Summary

SUMMARY

- Training a model means **minimising a cost function** (for a line, the mean squared error)
- Gradient descent: **new value = old value − learning rate × slope of the cost**, repeated
- The slope shrinks near the bottom, so the **steps shrink on their own**. Stop when the step is below about 0.001 or after about 1,000 steps
- The **learning rate** is chosen by you: too small is slow, too large diverges. Scale your features first
- **Batch** uses all rows per step, **stochastic** uses one random row, **mini-batch** uses a small random group (the usual default)
- SGD and mini-batch are faster per step and can learn from new data, at the price of a noisier path

Term	Meaning in one line
Parameter	A number the model learns (slope, intercept, weights)
Cost / loss function	One number measuring how wrong the predictions are
Gradient	The slope of the cost at the current parameter values
Learning rate	Small number that scales each step (you choose it)
Step size	Slope × learning rate
Epoch	One full pass through all the training data
Schedule	How the learning rate is lowered during training

Practice questions

Try each question before reading the answer underneath it.

Q1. Why do we subtract the slope instead of adding it?

> **Answer.** The slope points uphill. To go **down** the cost curve we move the opposite way, so we subtract (slope × learning rate).

Q2. The slope of the cost is −5.7 and the learning rate is 0.1. If the intercept is 0, what is the new intercept?

> **Answer.** Step size = −5.7 × 0.1 = −0.57. New intercept = 0 − (−0.57) = **0.57**.

Q3. Why do the steps get smaller as gradient descent approaches the minimum?

> **Answer.** The step size is slope × learning rate, and the slope gets closer to 0 near the bottom of the curve, so the step does too.

Q4. What goes wrong if the learning rate is too large? Too small?

> **Answer.** Too large: steps overshoot the minimum and the cost can grow instead of shrinking. Too small: it moves very slowly and may stop (maximum steps) before it gets there.

Q5. You have 10 million rows. Why might you prefer mini-batch gradient descent over batch?

> **Answer.** Batch needs a pass over all 10 million rows for **every** step. A mini-batch uses a small random subset, so each step is far cheaper, and the result is steadier than a single-sample step.

Q6. Name two stopping rules for gradient descent.

> **Answer.** Stop when the step size falls below a small number (about 0.001), and stop after a maximum number of steps (about 1,000).